

Introduction to Unix shell

Part 6

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Swiss army knives pt.2

- `awk`: Powerful programming language for text processing and data extraction
 - `awk` = **A**ho, **W**einberger, **K**ernighan (creators' names)
 - Originally created in 1977
 - Designed for processing structured text and tabular data
- Operates on input line by line, splitting each line into fields
- Basic usage: `awk [options] '<program>' <file>`
- Common options
 - `-F <delimiter>`: Specify field separator (default is whitespace)
 - `-v <var>=<value>`: Set variables before program execution
 - `-f <file>`: Read program from a file
- `awk` is both a command and a programming language with variables, functions, and control structures

- awk programs consist of patterns and actions
- Basic syntax: `pattern { action }`
 - If more actions needed: `pattern { action1; action2; ... }`
- Pattern types
 - **BEGIN**: Executed before processing any input
 - **END**: Executed after processing all input
 - **/regex/**: Execute action for lines matching the regex
 - **expression**: Execute action if expression is true
 - Empty pattern: Execute action for every line
- Variables and fields
 - **\$0**: The entire current line
 - **\$1, \$2, ...**: Individual fields (separated by field delimiter)
 - **NF**: Number of fields in the current line
 - **NR**: Number of records (lines) processed so far
 - User-defined variables: `var = value`

Variable	Description
FS	Field Separator (default: whitespace)
OFS	Output Field Separator (default: space)
RS	Record Separator (default: newline)
ORS	Output Record Separator (default: newline)
NR	Number of Records processed so far
NF	Number of Fields in current record
FILENAME	Name of current input file
FNR	File Record Number (line number in current file)
ARGC	Number of command-line arguments
ARGV	Array of command-line arguments

Basic awk Examples

```
# Print specific fields from a file
awk '{print $1, $3}' file.txt
# Print lines matching a pattern
awk '/error/ {print}' log.txt
# Print with line numbers
awk '{print NR ": " $0}' file.txt
# Print lines with more than 5 fields
awk 'NF > 5 {print}' file.txt
# Use custom field separator (colon)
awk -F: '{print $1}' /etc/passwd
# Print sum of values in a column
awk '{sum += $1} END {print "Sum: " sum}' numbers.txt
# Count lines matching a pattern
awk '/pattern/ {count++} END {print count}' file.txt
```

- Arithmetic operators: `+`, `-`, `*`, `/`, `%`, `^`
- Comparison operators: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Logical operators: `&&`, `||`, `!`
- String concatenation: Space between values (e.g., `$1 $2`)
- Pattern matching: `~` (match) and `!~` (not match)
 - Example: `$2 ~ /^[0-9]+$ /` checks if field 2 is numeric
- Ternary operator: `condition ? true_value : false_value`
- Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `^=`

- if-else statements
 - `if (condition) { action } else { other_action }`
- for loops
 - `for (i = 1; i <= 10; i++) { action }`
- while loops
 - `while (condition) { action }`
- do-while loops
 - `do { action } while (condition)`
- break and continue
 - `break`: Exit the loop
 - `continue`: Skip to next iteration
- `next`: Skip to next input record
- `exit`: Terminate the program

Function	Description
<code>length(string)</code>	Length of a string (or \$0 if no argument)
<code>substr(string, start, len)</code>	Extract substring starting at position <code>start</code>
<code>index(string, substr)</code>	Position of first occurrence of <code>substr</code>
<code>split(string, array, sep)</code>	Split string into array using separator
<code>sub(regex, repl, target)</code>	Replace first match of <code>regex</code> in <code>target</code>
<code>gsub(regex, repl, target)</code>	Replace all matches of <code>regex</code> in <code>target</code>
<code>match(string, regex)</code>	Check if string matches <code>regex</code>
<code>tolower(string)</code>	Convert string to lowercase
<code>toupper(string)</code>	Convert string to uppercase
<code>sprintf(format, ...)</code>	Format string (like C <code>printf</code>)

Function	Description
<code>int(number)</code>	Convert to integer
<code>sqrt(number)</code>	Square root
<code>sin(number), cos(number)</code>	Trigonometric functions
<code>rand()</code>	Random number between 0 and 1
<code>srand(seed)</code>	Set random seed
<code>system(command)</code>	Execute a system command
<code>getline</code>	Read next line from input
<code>close(file)</code>	Close a file opened by <code>getline</code> or <code>print</code>

String Functions Examples

```
# Extract substring
awk '{print substr($0, 1, 10)}' file.txt
# Find position of substring
awk '{print index($0, "error")}' log.txt
# Convert to uppercase
awk '{print toupper($0)}' file.txt
# Split line into array and process
awk '{split($0, arr, ":"); print arr[1]}' file.txt
# Replace first occurrence
awk '{sub(/old/, "new"); print}' file.txt
# Replace all occurrences
awk '{gsub(/old/, "new"); print}' file.txt
# String length
awk '{print length($0)}' file.txt
# Format output with sprintf
awk '{printf "Line %d: %s\n", NR, $0}' file.txt
```

```
# Count occurrences of each word
awk '{for(i=1;i<=NF;i++) cnt[$i]++} END {for(w in cnt) print w, cnt[w]}' file.txt

# Sum column by category
awk '{sum[$1] += $2} END {for(cat in sum) print cat, sum[cat]}' data.txt

# Calculate average of a column
awk '{sum += $1; count++} END {print "Average: " sum/count}' numbers.txt

# Print only lines where field value exceeds threshold
awk '$3 > 100 {print $1, $3}' data.txt

# Conditional processing with multiple fields
awk '$2 == "error" && $3 > 5 {print "High error: " $0}' log.txt

# Use regex in condition
awk '$1 ~ /^[A-Z]/ {print}' file.txt

# Print fields in reverse order
awk '{for(i=NF; i>=1; i--) printf "%s ", $i; print ""}' file.txt
```

Processing CSV Files with awk

```
# Extract specific columns from CSV (comma-separated)
awk -F, '{print $1, $3}' data.csv
# Sum values in a CSV column
awk -F, '{sum += $2} END {print "Total: " sum}' data.csv
# Filter CSV rows based on condition
awk -F, '$3 > 50 {print}' data.csv
# Count records by category (CSV)
awk -F, '{count[$1]++} END {for(cat in count) print cat, count[cat]}' data.csv
# Extract and reformat CSV data
awk -F, '{printf "ID: %s, Name: %s\n", $1, $2}' data.csv
# Remove duplicate rows (based on first field)
awk -F, '!seen[$1]++ {print}' data.csv
# Join CSV fields with different separator in output
awk -F, '{print $1 "|" $2 "|" $3}' data.csv
```

Log File Analysis with awk

```
# access.log combined format:
# client_ip ident user [time] "request_line" status_code size referrer user_agent
# Count HTTP status codes (status code is 6th field)
awk '{print $6}' access.log | sort | uniq -c
# Extract and count status codes better solution
awk '{cnt[$6]++} END {for(status in cnt) print status, cnt[status]}' access.log
# Count requests per IP address
awk '{count[$1]++} END {for(ip in count) print ip, count[ip]}' access.log
# Extract URLs from Apache logs (URL is 5th field)
awk '{print $5}' access.log | sort | uniq -c | sort -rn
```

- awk allows definition of custom functions
- Basic syntax:
`function name(param1, param2, ...) { actions; return value }`
- Functions must be defined before they are used
- Functions can have local variables (declared as extra parameters)
- Return statement is optional; functions return 0 or empty string by default
- Typical use case: Reusable calculations or string processing

User-defined Functions in awk

```
function celsius_to_fahrenheit(c) {  
    return c * 9/5 + 32  
}  
  
function is_even(n) {  
    return n % 2 == 0  
}  
  
BEGIN {  
    print "0C = " celsius_to_fahrenheit(0) "F"  
    print "25C = " celsius_to_fahrenheit(25) "F"  
  
    for(i=1; i<=10; i++) {  
        if(is_even(i))  
            print i " is even"  
    }  
}
```


Complex awk Script Example

```
BEGIN {  
    FS=","  
    OFS=" | "  
    total=0  
    count=0  
}  
  
NR > 1 { # Skip header  
    price=$3  
    quantity=$4  
    subtotal = price * quantity  
    total += subtotal  
    count++  
  
    printf "%s %s: %.2f x %d = %.2f\n",  
        $1, $2, price, quantity, subtotal  
}
```

```
END {  
    if(count > 0) {  
        avg = total / count  
        print "=====  
        printf "Total: %.2f\n", total  
        printf "Average: %.2f\n", avg  
        printf "Count: %d items\n", count  
    }  
}
```

Feature	sed	grep	awk
Text substitution	Excellent	Basic	Good
Pattern matching	Good	Excellent	Good
Line deletion	Excellent	Limited	Good
Structured data	Limited	Limited	Excellent
Field extraction	Limited	Limited	Excellent
Complexity	Low	Low	Medium

- The `find` command searches for files and directories in a directory hierarchy
- Powerful tool for locating files based on various criteria (name, size, modification time, permissions, etc.)
- Basic usage: `find [path] [options] [expression]`
- If no path is specified, the current directory is used
- `find` recursively searches subdirectories
- Returns matching file paths, one per line
- Common use cases
 - Finding files by name or pattern
 - Locating files by size or age
 - Finding files with specific permissions
 - Batch processing files with `-exec` or `-print0`

- `-name <pattern>`: Find files matching a name pattern
 - Example: `find . -name "*.txt"` finds all .txt files
- `-iname <pattern>`: Case-insensitive name matching
- `-type <type>`: Find files of specific type
 - `f` = regular file, `d` = directory, `l` = symbolic link, etc.
- `-size <size>`: Find files by size
 - Example: `find . -size +1M` finds files larger than 1 MB
- `-mtime <days>`: Find files modified within specified days
 - Example: `find . -mtime -7` finds files modified in the last 7 days
- `-perm <mode>`: Find files with specific permissions
- `-user <username>`: Find files owned by specific user
- `-group <groupname>`: Find files owned by specific group

- `-path <pattern>`: Match full path (not just filename)
 - Example: `find . -path "*/backup/*"` finds files in backup directories
- `-regex <pattern>`: Match against regular expression
- `-maxdepth <levels>`: Limit search depth
 - Example: `find . -maxdepth 2 -name "*.log"` searches only 2 levels deep
- `-mindepth <levels>`: Start search at specified depth
- `-newer <file>`: Find files modified after another file
- `-empty`: Find empty files or directories
- `-executable`: Find files that are executable
- `-readable`, `-writable`: Find files with specific permissions for current user

Combining Conditions in find

- Multiple conditions can be combined using logical operators
- `-and` (or implicit): Both conditions must be true
 - `find . -name "*.log" -and -size +10M`
- `-or`: Either condition must be true
 - `find . -name "*.log" -or -name "*.txt"`
- `-not` or `!`: Negate a condition
 - `find . -not -name "*.tmp"` finds files that are NOT .tmp files
- Parentheses can group conditions
 - `find . \( -name "*.log" -o -name "*.txt" \) -size +1M`
 - **NOTE:** Parentheses must be escaped with backslash in the shell

- `-exec <command> \;`: Execute command on each found file
 - `{}` is replaced with the filename
 - Example: `find . -name "*.log" -exec rm {} \;`
- `-exec <command> \+`: More efficient; passes multiple files to command
 - Example: `find . -name "*.log" -exec rm {} \+`
- `-ok <command> \;`: Like `-exec` but prompts for confirmation
- `-print`: Print the filename (default action)
- `-print0`: Print filenames separated by null character
 - Useful for handling filenames with spaces or special characters
 - Works with `xargs -0`

Basic find Examples

```
# Find all .txt files in current directory and subdirectories
find . -name "*.txt"
# Find all files larger than 100 MB
find . -size +100M
# Find all directories (not files)
find . -type d
# Find files modified in the last 7 days
find . -mtime -7
# Find all empty files
find . -empty
# Find files owned by a specific user
find . -user username
# Find files with specific extension in a specific directory
find /home/user -name "*.pdf"
# Find files NOT matching a pattern
find . -not -name "*.tmp"
```


Advanced find Examples

```
# Find and delete files matching a pattern
find . -name "*.log" -exec rm {} \;
# Find and move files to another directory
find . -name "*.bak" -exec mv {} /backup/ \;
# Find large files and list them with size
find . -size +50M -exec ls -lh {} \;
# Find recently modified files and display them
find . -mtime -1 -exec echo "Modified: {}" \;
# Count number of .txt files
find . -name "*.txt" | wc -l
# Find files and search for specific content
find . -name "*.conf" -exec grep -l "error" {} \;
# Combine conditions: find .conf files larger than 10KB modified in last 30 days
find . -name "*.conf" -size +10k -mtime -30
```

- Filenames with spaces can cause issues when piping `find` output to other commands

- Problem example

```
find . -name "*.txt" | xargs rm
```

If a file is named “my file.txt”, it will be treated as two separate files

- Solution: Use `-print0` with `xargs -0`
 - `-print0`: Terminates filenames with null character instead of newline
 - `xargs -0`: Reads null-separated input

- Safe approach:

```
find . -name "*.txt" -print0 | xargs -0 rm
```

- Better approach: Use `-exec` directly
 - `find . -name "*.txt" -exec rm {}`

Comparison of Approaches

```
#!/bin/bash
# Create test files with spaces
touch "my file.txt" "another file.log"

# UNSAFE: Does not handle spaces properly
find . -maxdepth 1 -name "*.txt" | xargs rm -v
rm -v $(find . -maxdepth 1 -name "*.txt")
# These both fails: "my" and "file.txt" treated as separate files

# SAFE: Using -print0 with xargs -0
find . -maxdepth 1 -name "*.txt" -print0 | xargs -0 rm -v

# SAFE: Using -exec (recommended)
find . -maxdepth 1 -name "*.log" -exec rm -v {} \;

# SAFE: Using -exec with + (most efficient for multiple files)
find . -maxdepth 1 -type f -exec rm -v {} \+
```

Why Spaces Cause Problems

- Shell interprets whitespace as field separator
- When `find` outputs “my file.txt”, and it’s piped through `xargs`:
 - `xargs` sees two separate arguments: “my” and “file.txt”
 - Commands receive both as separate parameters
- Null character (`\0`) is a special character
 - Cannot appear in filenames
 - Guaranteed to be a safe delimiter
 - Both `find -print0` and `xargs -0` understand null-separated data
- `-exec` avoids the issue entirely
 - Directly passes filename to command
 - No shell word-splitting involved
 - Slightly less efficient but very safe

```
# Find and count lines in all .txt files (handles spaces)
find . -name "*.txt" -print0 | xargs -0 wc -l
# Find and compress files (handles spaces in filenames)
find . -name "*.log" -print0 | xargs -0 gzip
# Find and change permissions on files with spaces
find . -type f -name "*.sh" -print0 | xargs -0 chmod +x
# Find large files and list them with detailed info (handles spaces)
find . -size +100M -exec ls -lh {} \;
# Search for specific content in files with spaces in names
find . -type f -name "*.conf" -exec grep -H "password" {} \;
# Count total size of matching files (handles spaces)
find . -name "*.tmp" -print0 | xargs -0 du -c | tail -1
```

Lab experience

- Already done exercise: Given a web server `access.log` file count the number of requests that are successful (200) or wrong (4xx and 5xx) using `awk`
 - Refer to the exercise that used `grep` and `wc`
- Get CPU load from `/proc/stat` using `awk`
 - Refer to the exercise on custom OS Health check in part 4

Write a shell script that accepts a variable number of parameters, N :

- The first parameter is the name of a directory that identifies a hierarchy G in the file system.
- The remaining $N - 1$ parameters are single characters $(C_1, \dots, C_{N-1})$.

After validating the parameters, the program must explore the hierarchy G and must count **globally** all the files that satisfy the following specification:

- The files must be readable.
- They must contain (within their content) at least one occurrence of **at least one** of the characters $(C_1, \dots, C_{N-1})$.

For each analyzed directory, its name must be printed to the standard output, preceded by the character $+$ if at least one file satisfying the above specification was found inside it; otherwise, it must be preceded by $-$.

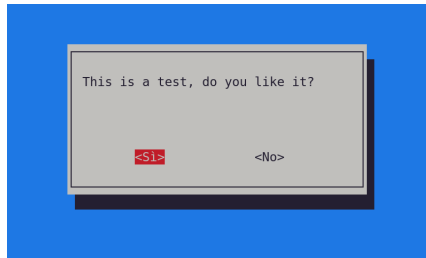
At the end, the total number of files found (that satisfy the condition stated above) must be printed to the standard output.

Example output

```
- /home/user/example/  
+ /home/user/example/subdir1  
- /home/user/example/subdir2  
Total files: 3
```

Bonus tips and tricks

- Improving user experience
 - Controls like dialogs, entryfields, ...
 - Alternative to `read -p`
- Several tools to create useful UI controls
 - `dialog`: the classical tool
 - `whiptail`: drop-in replacement for `dialog`
- Interaction with GUI
 - `zenity`: GNOME-based tool. Integrates support for file chooser, progress bar, notification icons and messages



- Displaying simple status notifications
 - `--msgbox`: Renders an alert with a mandatory “OK” button
 - Halts execution flow until the user acknowledges the window
- Non-blocking information indicators
 - `--infobox`: Displays status strings instantaneously without pausing
 - Often paired with background tasks and short `sleep` intervals

Examples

- Message Box

```
whiptail --title "Success" \  
--msgbox "Setup OK!" 8 45
```

- Info Box

```
dialog \  
--infobox "Syncing DB" 8 45
```

- Handling branching logic queries
 - `--yesno`: Creates a dialog presenting “Yes” and “No” buttons
- Native shell signal integration
 - Leverages terminal exit status codes (`$?`) for control flow
 - Return code 0 represents “Yes”
 - Return code 1 represents “No”
 - Return code 255 signals an ESC cancel exit

Shell Branching Control Flow:

```
if whiptail --yesno "Apply updates?" 8 45
then
    echo "Upgrading packages..."
else
    echo "Upgrade canceled."
fi
```

- Adjusting button text for better context
 - dialog uses `--yes-label` and `--no-label`
 - whiptail uses `--yes-button` and `--no-button`
- Altering default focus
 - Add `--defaultno` to highlight the negative option by default (prevents accidental destructive actions)
- Modernizing terminal confirmations
 - Clearer call-to-actions (e.g., “Install/Abort” instead of generic “Yes/No”)

Custom dialog example

```
whiptail --title "Confirmation"
--yes-button "Install"
--no-button "Abort"
--defaultno
--yesno "Proceed with setup?" 8 45
```

- Prompting for user configuration input
 - `--inputbox`: Provides a standard input line field
 - `--passwordbox`: Hides character displays to mask inputs safely
- Overcoming the stream obstacle
 - Since interactive components render on `stdout`, string inputs are printed to `stderr`
 - Stream file descriptors must be swapped to successfully assign user inputs to shell variables

Capturing Stream Inputs

- File descriptor mapping

```
USER=$(whiptail \  
  --inputbox "Username" 8 45 \  
  3>&1 1>&2 2>&3)
```
- Explanation
 - `3>&1` duplicates stream 3 to standard output
 - `1>&2` directs standard output to standard error
 - `2>&3` maps the string output from standard error to standard output

- Displaying single-choice options
 - `--menu`: Offers interactive vertical item selection lists
- Structural parameter arrays
 - Layout bounds: `<height> <width>`
`<menu-height>`
 - Item pairing values require a *tag* followed by an item *description*
 - Returns only the selected option's short *tag* to standard error

Shell Menu Definition

- Interactive menu structure

```
CHOICE=$(whiptail --menu "DB" 12 45 3 \  
    "1" "PostgreSQL" \  
    "2" "MariaDB" \  
    "3" "SQLite" \  
    3>&1 1>&2 2>&3)
```


- Visualizing long-running tasks
 - `--gauge`: a progress bar
 - Accepts layout dimensions and initial percentage
- Dynamic status feeding
 - Reads progress values (0 to 100) continuously from standard input (`stdin`)
 - Automatically updates when a new integer line is received
- Stream termination
 - Closes and returns control once standard input stream hits EOF (End of File)

Dynamic Gauge Piping:

- Feeder command loop

```
{  
for i in 20 50 80 100; do  
    echo $i; sleep 1  
done  
} | whiptail --title "Loading" \  
    --gauge "Please wait..." 8 45 0  
# grouping with {} is not necessary.  
# Anything within the {} is like  
# a single command
```